11/15

INDIAN INSTITUTE OF TECHNOLOGY MADRAS

A

Roll No. | I | D | 2 | 5 | S | 0 | 2 | 7 |

Name : PUNEET

Total No. of Pages

Quiz I ☐ Quiz II/ Mid-Sem ☐ End-Semester ☐ Make-up ☐ Date : 21/10/2026

Semester & Degree : II, MS Course No. MA5105 Part :

Question No.	1	2	3	4	5	6	7	8	9	10
Marks										

11	12	13	14	15	16	17	18	19	20	Total

Answer on both sides of the paper including the space below

1. $f: R^n \to R$

min f

$x_{k+1} = x_k - \alpha \nabla f(x_k)$

$\alpha > 0$, step size

Pseudocode for steepest descent algorithm

Initialize
starting point x_0
tolerance ε, $\varepsilon > 0$
step size α, $\alpha > 0$
max_iter, $k = 0$, iter $= 0$ } (a)

while True :

compute the gradient
$\nabla f(x_k)$ } (b)

~~if $\|\nabla f(x_k)\|_2 < \varepsilon$~~

break

iter = iter + 1

$x_{k+1} = x_k - \alpha \nabla f(x_k)$ } d

c { if iter == max_iter or
$\|\nabla f(x_k)\|_2 < \varepsilon$,

e { break
else:
continue. (5)

return x_k.

2. $\min_{(x,y)\in R^2} f(x,y) = (x-2)^2 + (y-1)^2$

constraint $x + y = 2$

Penalized objective func^n,

$F_\mu(x,y) = f(x,y) + \frac{\mu}{2}(x+y-2)^2$

$\mu > 0$

Initialize
Points (x_0, y_0)
Penalty Parameter μ, $\mu > 0$
Tolerance ε, $\varepsilon > 0$
max_outer_iterations,
iter = 0

a

helper func^n for b

```
def func(x, y):
    return ((x-2)^2 + (y-1)^2)
```

b

```
def penal_obj_func^n(x, y):
    return (func(x, y) + (μ/2)(x+y-2)^2)
```

```
def steepest_descent_method(x_0, ε, α, max_iter)
    x_0 = 0
    ε = 0
    α = 0
    k = 0
    iter = 0
    while true:
        compute gradient ∇f(x_k)
        iter += 1
        x_{k+1} = x_k − α ∇f(x_k)
        if iter == max_iter or ||∇f(x_k)||_2 < ε
            break
        else continue
    return x_k
```

Over iterating for penalty function?

c

Minimiser of f is s.t. that $f' = 0$.

∴,

using steepest_descent_method for F_μ

equate $F'_\mu = 0$

and get the minimum F_μ

this means $f'(x,y) + \mu(x+y-2) = 0$

d { if $(x+y-2) < \varepsilon$: break

else :

e { $\mu = 10\mu$

repeat the minimization procedure

iter += 1

if $(x+y-2) < \varepsilon$ or iter == max_iter :

break

else :

continue

(f) The role of μ is to help convert a constrained optimization problem into an unconstrained optimization problem.

(2) The penalty parameter μ helps in adding an error term so while minimizing the objective func^n becomes smooth. Square value of it?

The term $(x+y-2)$ acts as the violation check for knowing whether convergence has been hit or not.

This is from the standpoint of code. I am not sure, how it will help mathematically.